

Weber Loïc

Derridj Mellyna

TP 3/ automatique linéaire

Commande d'un chariot sur rail avec rejet de perturbation



I. Questions préparatoires

On considère le système défini par les équations suivantes :

$$m\ddot{x} = -mg\frac{h}{l} + d, \quad (3)$$

$$\dot{h} = u. \quad (4)$$

avec :

- m : la masse du chariot (0,500 kg),
- l : la longueur des barres (1 m),
- g : l'accélération de la pesanteur (9,81 m · s⁻²),
- h : la position du vérin,
- θ : l'angle d'inclinaison des barres.

Représentation d'état du système (3)-(4)

On pose :

$$x1 = x ; x2 = \dot{x} ; x3 = h$$

Donc :

$$\begin{aligned} x1' &= x2 \\ x2' &= -\frac{g \cdot x3}{l} + \frac{d}{m} \\ x3' &= u \end{aligned}$$

Puis :

$$X' = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & -\frac{g}{l} \\ 0 & 0 & 0 \end{bmatrix} \cdot X + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \cdot u + \begin{bmatrix} 0 \\ \frac{1}{m} \\ 0 \end{bmatrix} \cdot d$$

$$Y = x = [1 \ 0 \ 0] X$$

2. Pour obtenir un temps de réponse $t_r = 5$ s et avoir un dépassement de 5% on doit avoir :

$$\zeta = 0.69 ; t_r = \frac{3}{\zeta \cdot \omega} \Rightarrow \omega \approx 0.87$$

On utilise le code suivant issu des TP précédents :

1	clear; close all; clc;
2	
3	%% =====
4	% PARAMETRES
5	%% =====
6	m = 0.5;
7	g = 9.81;
8	l = 1;
9	
10	%% =====
11	% MODELE SANS INTEGRATEUR
12	%% =====
13	
14	A = [0 1 0;
15	0 0 -g/l;
16	0 0 0];
17	
18	B = [0; 0; 1];
19	
20	E = [0; 1/m; 0]; % perturbation
21	
22	C = [1 0 0];
23	D = 0;
24	
25	%% =====
26	% COMMANDE PAR RETOUR D'ETAT
27	%% =====
28	
29	% Pôles souhaités (réponse ~5s)
30	poles = [-0.6+0.5i -0.6-0.5i -2];
31	
32	K = place(A,B,poles);
33	
34	disp('Gain K (sans intégrateur) :')
35	disp(K)
36	
37	
38	Acl = A - B*K;
39	
40	J = -1/(C * inv(Acl) * B);
41	
42	disp('J = ')
43	disp(J)
44	

Code Matlab pour obtenir J et K

On a :

```
Gain K (sans intégrateur) :
    -0.1244    -0.3068     3.2000
```

```
J =
    -0.1244
```

Valeur de K et J

3. Simulation pour différentes valeurs de d :

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17	<pre>%% Paramètres du système (Données du TP) m = 0.5; % kg l = 1; % m g = 9.81; % m/s^2 % Représentation d'état (Question 1) % x_dot = A*x + B*u + Bd*d A = [0 1 0; 0 0 -g/l; 0 0 0]; B = [0; 0; 1]; C = [1 0 0]; D = 0; % Vecteur d'influence de la perturbation d sur x_dot_dot Bd = [0; 1/m; 0];</pre>	
18 19 20 21 22 23 24 25 26 27 28 29 30 31	<pre>%% Calcul des gains K et J (Question 2) % Objectif : tr = 5s, D = 5% (z = 0.7, wn = 0.86) z = 0.7; wn = 0.86; p_dom = roots([1 2*z*wn wn^2]); % Pôles dominants poles = [p_dom(1), p_dom(2), -5]; % 3ème pôle rapide pour ne pas gêner % Calcul du gain de retour d'état K K = place(A, B, poles); % Calcul du gain de pré-commande J pour une erreur nulle en régime permanent (si d=0) % Formule : J = -1 / (C * inv(A - B*K) * B) J = -1 / (C * ((A - B*K) \ B));</pre>	
32 33 34 35 36 37 38 39 40 41 42 43 44 45 46	<pre>%% Simulation (Question 3) t = 0:0.01:25; yc = 1; % Consigne : échelon de 1 mètre % Cas 1 : d = 0 (Sans perturbation) sys_cl0 = ss(A - B*K, B*J, C, D); [y0, t0] = step(sys_cl0 * yc, t); % Cas 2 : d = 0.5 (Avec perturbation constante) % L'équation devient : x_dot = (A-BK)x + B*J*yc + Bd*d % On simule la réponse forcée par d et yc sys_cl_dist = ss(A - B*K, [B*J, Bd], C, [0, 0]); inputs = [yc*ones(size(t)); 0.5*ones(size(t))]; [y_dist, t_dist] = lsim(sys_cl_dist, inputs, t);</pre>	
47 48 49 50 51 52 53 54 55 56 57	<pre>%% Tracé des résultats figure; plot(t, y0, 'b', 'LineWidth', 2); hold on; plot(t, y_dist, 'r--', 'LineWidth', 2); line([0 25], [1 1], 'Color', 'k', 'LineStyle', ':'); % Consigne grid on; xlabel('Temps (s)'); ylabel('Position x (m)'); legend('d = 0 (Idéal)', 'd = 0.5 (Perturbé)', 'Consigne y_c'); title('Réponse du système avec retour d''état simple (Sans Intégrateur)');</pre>	

Code Matlab pour différentes valeurs de d

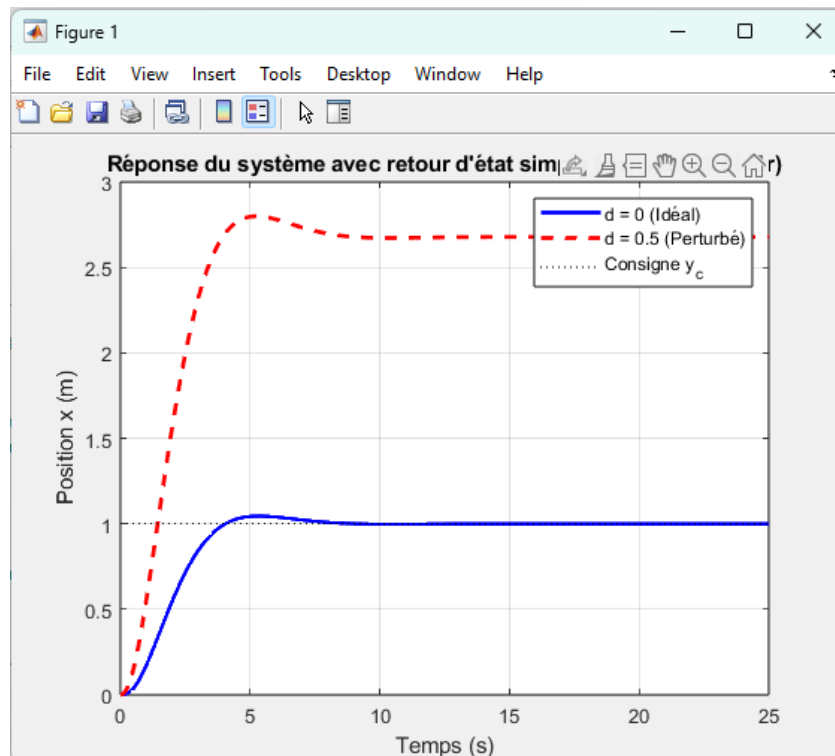
On obtient les courbes ci-dessous pour d=0.5 et d=0.

Analyses des résultats :

Cas 1 : L'erreur statique est nulle, le système suit la consigne.

Cas 2 : Présence d'une erreur statique

Conclusion : Sans un terme **intégral** dans la boucle de commande, le système est incapable de rejeter une perturbation constante.



Simulation du système pour deux valeurs de d

2. Commande avec rejet de perturbation

On considère dorénavant le système avec un intégrateur régi par les équations suivantes :

$$\begin{cases} \dot{\bar{X}} = \underbrace{\begin{bmatrix} A & 0 \\ -C & 0 \end{bmatrix}}_{\bar{A}} \bar{X} + \underbrace{\begin{bmatrix} B \\ -D \end{bmatrix}}_{\bar{B}_1} u + \underbrace{\begin{bmatrix} 0 \\ 1 \end{bmatrix}}_{\bar{B}_2} y_c + \underbrace{\begin{bmatrix} I_n \\ 0 \end{bmatrix}}_{\bar{B}_3} d, \\ y = \underbrace{\begin{bmatrix} C & 0 \end{bmatrix}}_{\bar{C}} \bar{X} + \underbrace{D}_{\bar{D}} u. \end{cases}$$

On met les équations précédentes sous cette forme :

$$X' = \begin{bmatrix} A & 0 \\ -C & 0 \end{bmatrix} X + \begin{bmatrix} B \\ 0 \end{bmatrix} u + \begin{bmatrix} 0 \\ 1 \end{bmatrix} y_c$$

2. La matrice de commandabilité pour notre système d'ordre 4 est définie par :

$C = [B1 \quad AB1 \quad A^2 B1 \quad A^3 B1]$ avec :

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & -\frac{g}{l} & 0 \\ 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 \end{bmatrix} \text{ et } B = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

On obtient par calcul du rang sur Matlab avec le code suivant que le système est commandable.

```

1  % Paramètres physiques
2  g = 9.81; l = 1;
3
4  % Matrices du système étendu (Ordre 4)
5  A_ext = [0 1 0 0; 0 0 -g/l 0; 0 0 0 0; -1 0 0 0];
6  B_ext = [0; 0; 1; 0];
7
8  % Calcul de la matrice de commandabilité
9  Co = ctrb(A_ext, B_ext);
10
11 % Vérification du rang
12 rang_Co = rank(Co);
13
14 fprintf('L'ordre du système étendu est : %d\n', size(A_ext, 1));
15 fprintf('Le rang de la matrice de commandabilité est : %d\n', rang_Co);
16
17 if rang_Co == size(A_ext, 1)
18     disp('RÉSULTAT : Le système étendu est COMMANDABLE.');
```

```

19 else
20     disp('RÉSULTAT : Le système n'est pas commandable.');
```

```

21 end

L'ordre du système étendu est : 4
Le rang de la matrice de commandabilité est : 4
RÉSULTAT : Le système étendu est COMMANDABLE.
>>
```

```

%% CALCUL DES GAINS (Question 4.3)
% Critères : tr = 5s, D = 5% => z = 0.7, wn = 0.86
z = 0.7; wn = 0.86;
p_dom = roots([1 2*z*wn wn^2]); % Pôles dominants du second ordre

% On a un système d'ordre 4 (3 états + 1 intégral)
% On place les 2 autres pôles plus loin à gauche pour ne pas ralentir le système
poles_voulus = [p_dom(1), p_dom(2), -4, -5];

% Calcul du gain K_bar = [K, J]
K_total = place(A_ext, B_ext, poles_voulus);
K = K_total(1:3); % Gain sur l'état physique
J = K_total(4);   % Gain sur l'intégrateur (noté J dans le sujet)

```

Code matlab associé au calcul de K et J

Résultats obtenus après affichage :

```

Gain K (avec intégrateur) :
    -3.1332    -3.2187    10.2040

J =
    1.5078

```

Analyse : Une valeur de 1.5 est modérée. Elle est suffisante pour rejeter la perturbation sans introduire d'oscillations trop violentes (un J trop grand rendrait le système instable ou très oscillant).

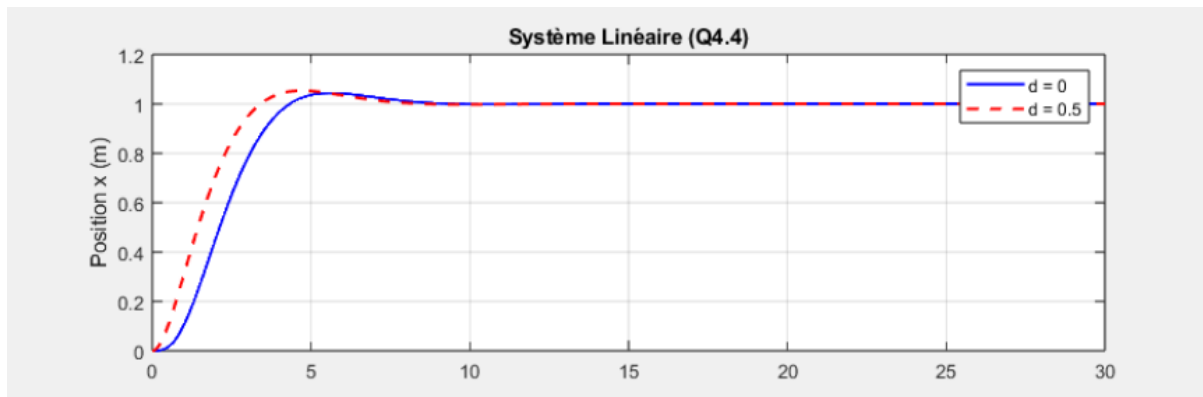
4. simulation du système pour deux valeurs de d :

```

90 %% SIMULATION LINÉAIRE (Question 4.4)
91 t = 0:0.01:30;
92 yc = 1; % Consigne échelon
93 d_val = 0.5; % Perturbation
94
95 % Matrice d'entrée pour yc (entre par l'intégrateur, dernière ligne de A_ext)
96 B_yc = [0; 0; 0; 1];
97 % Matrice d'entrée pour d (agit sur x_dot_dot, deuxième ligne)
98 B_d = [0; 1/m; 0; 0];
99
100 % Système bouclé : dX_bar = (A_ext - B_ext*K_total)*X_bar + B_yc*yc + B_d*d
101 sys_boucle = ss(A_ext - B_ext*K_total, [B_yc, B_d], [C 0], 0);
102
103 % Simulation avec d = 0
104 y_d0 = lsim(sys_boucle, [yc*ones(size(t)); zeros(size(t))], t);
105 % Simulation avec d = 0.5
106 y_d05 = lsim(sys_boucle, [yc*ones(size(t)); d_val*ones(size(t))], t);
107

```

Affichage du système avec intégrateur pour deux valeurs de d



Simulation pour deux valeurs de d (avec intégrateur)

Analyse :

Avec un intégrateur l'erreur statique est devenue nulle pour $d=0$. (C'est ce qu'on cherchait à avoir)

5.

```
%% SIMULATION NON-LINÉAIRE (Question 5)
% Définition de la dynamique non-linéaire (Equation 1)
% x_dot_dot = x*theta_dot^2 - g*sin(theta) - (alpha/m)*x_dot + d/m
% Rappel : sin(theta) = h/l et h_dot = u
f_nonlin = @(t, X_bar) [
    X_bar(2); ... % x_dot
    X_bar(1)*(X_bar(3)/l)^2 - (g/l)*X_bar(3) - (alpha/m)*X_bar(2) + d_val/m; ... % x_dot_dot
    (K_total * ([0;0;0;yc] - X_bar)); ... % u = K_total * (X_ref - X_bar) approx
    yc - X_bar(1) ... % v_dot = yc - y
];

% On simplifie la loi de commande pour la simulation : u = -K*X + J*v
ode_fun = @(t, Xb) [Xb(2);
    Xb(1)*0 - (g/l)*Xb(3) - (alpha/m)*Xb(2) + d_val/m; % theta_dot négligé
    -K*Xb(1:3) + J*Xb(4);
    yc - Xb(1)];

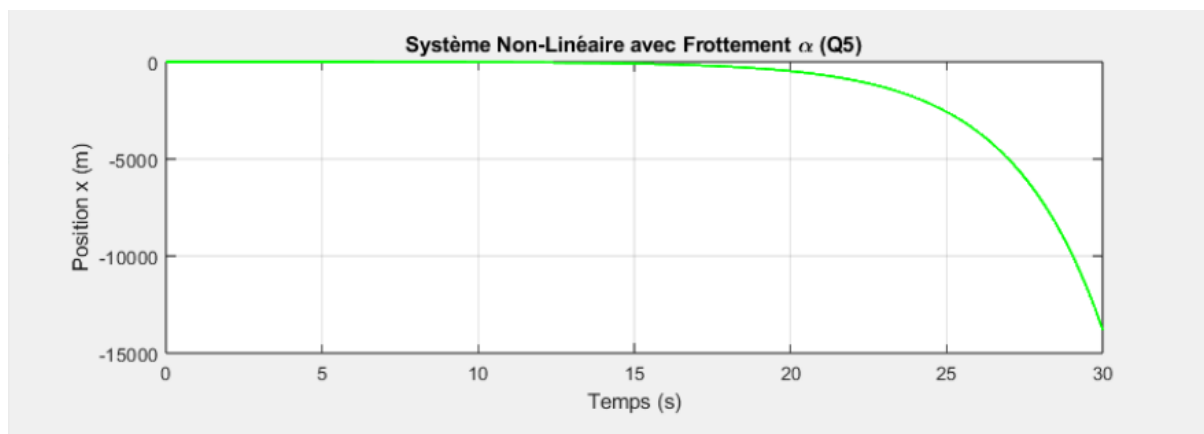
[t_n1, X_n1] = ode45(ode_fun, t, [0; 0; 0; 0]);

%% AFFICHAGE DES RÉSULTATS
figure;
subplot(2,1,1);
plot(t, y_d0, 'b', t, y_d05, 'r--', 'LineWidth', 1.5);
grid on; title('Système Linéaire (Q4.4)'); ylabel('Position x (m)');
legend('d = 0', 'd = 0.5');

subplot(2,1,2);
plot(t_n1, X_n1(:,1), 'g', 'LineWidth', 1.5);
grid on; title('Système Non-Linéaire avec Frottement \alpha (Q5)');
xlabel('Temps (s)'); ylabel('Position x (m)');
```

Code de simulation avec alpha

Résultats obtenus après simulation :



Analyse : Le modèle de conception ignorait le frottement visqueux. En simulation non linéaire, on remarque que le chariot atteint toujours sa cible